

CS 322: Software Construction

□ Course Teacher: Dr. Muddassar Azam Sindhu

«#»

1

Static Checking

- Today we will learn about:
 - Static Checking
 - Three essential properties of good software

«#»

2

Hailstone Sequence

- The hailstone sequence is defined as under:
 - Starting with a number 'n', the next number in the sequence is $n/2$ if n is even, or $3n+1$ if n is odd. The sequence ends when n reaches 1.
 - Examples:
 - 2, 1
 - 3, 10, 5, 16, 8, 4, 2, 1
 - 4, 2, 1
 - 7, 22, 11, 34, 17, 52, 26, 13, 40, ... (Will this stop/terminate?)
 - Due to the odd number rule, the sequence may bounce up and down.
 - It's conjectured that all hailstones eventually fall to the ground.
 - Open question (shown till 2.36×10^{21})

<#>

3

Computing Hailstones

- Java and Python codes for Computing Hailstone Sequences for some starting n.

```
// Java
int n = 3;
while (n != 1) {
    System.out.println(n);
    if (n % 2 == 0) {
        n = n / 2;
    } else {
        n = 3 * n + 1;
    }
}
System.out.println(n);
```

```
# Python
n = 3
while n != 1:
    print n
    if n % 2 == 0:
        n = n / 2
    else:
        n = 3 * n + 1
print n
```

<#>

4

Computing Hailstones Cont'd

- Few things to note:
 - Basic semantics of both Java and Python statements are very similar like, **if** and **while**.
 - Java requires a semicolons at the end of statements which can be restrictive but gives freedom to organize code.
 - Parentheses in Java around conditions of **if** and **while**
 - Java requires curly braces around blocks
 - Blocks should be indented (readability matters not just for compilers but also for humans. Why?)

<#>

5

Types

- The most important difference between Java and Python is the declaration of variable **n**. Java specifies its type as **int**.
- A type is a **set of values**, along with **operations** that can be performed on those values.
- Java has several primitive types, among them:
 - **int** (for integers but in the range $\pm 2^{31}$ or roughly 2 billion)
 - **long** (for larger integers upto $\pm 2^{63}$)
 - **boolean** (for true and false)
 - **double** (for floating-point numbers which represent a subset of real numbers)
 - **char** (for single characters)

<#>

6

Object Types

- Java also has object types, like:
 - `String`
 - Represents a sequence of characters
 - `BigInteger` represents an integer of arbitrary size.
- By convention primitive types are lower case while object types start with a capital letter.
- Operations are functions that take inputs and produce outputs (and sometimes change the values themselves). Syntax for operations varies but we still think of them as functions.
 - As an infix, prefix or postfix operator. `a + b` invokes the operation `+`: `int x int → int`
 - As a method of an object. E.g `bigint1.add(bigint2)` calls `add: BigInteger x BigInteger → BigInteger`
 - As a function. E.g., `Math.sin(theta)` calls the operation `sin: double → double`. Here `Math` is not an object but the name of a class which contains the `sin` function.
 - Contrast Java's `str.length()` and Python's `len(str)`?
- Overloaded Operations?

<#>

7

Static Typing and Dynamic Typing

- Java is a statically-typed language. In such languages the types of all variables are known at compile time (i.e., before the program runs) and the compiler can deduce the types of expressions as well.
 - If `a` and `b` are declared as `int`, the compiler deduces that `a + b` will also be of type `int`.
 - This helps in detecting many errors by the compiler/IDE.
- In dynamically-typed languages like Python this kind of checking is deferred till runtime.
- Static Typing is a special type of **static checking**, which means checking for bugs at compile time.
- Bugs are a bane of programming and this course aims to equip you with ideas to eliminate bugs from your code.
 - Static checking being the first such idea.
 - `"5" * "6"`

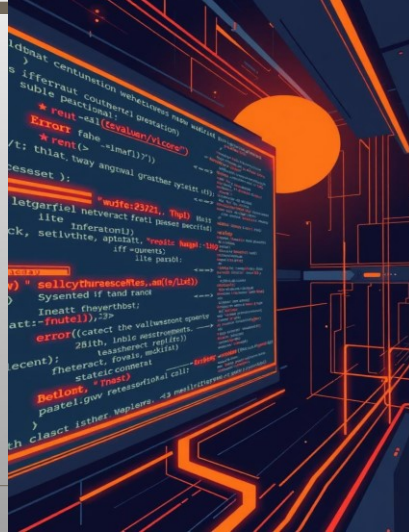
<#>

8

The Power of Checking: Static, Dynamic, and Beyond

- ❑ Understanding how programming languages catch errors
- ❑ Three kinds of automatic checking:
 - Static Checking
 - ❑ Bug found before program runs.
 - Dynamic Checking
 - ❑ Bug found during execution.
 - No checking
 - ❑ Language offers no help, manual vigilance needed.
- ❑ Catching bugs statically is often preferable
 - Why?

<#>



9

What Static Checking Catches

- ❑ Syntax errors
 - Extra punctuation, indentation, spurious words
- ❑ Name & Argument Errors
 - Wrong names like (Math.sine), incorrect / wrong type or no. of arguments
- ❑ Return type mismatches
 - E.g., function returning a string instead of an int.

<#>

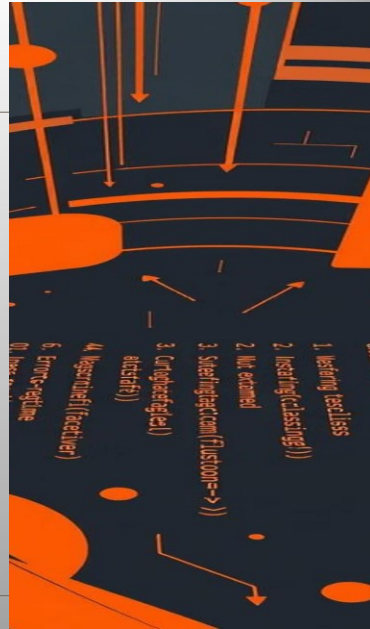


10

What Dynamic Checking catches?

- ☐ Illegal Argument Values
 - Errors dependent on specific values like divide-by-zero
- ☐ Out of range indices
 - Accessing array / string with wrong index
- ☐ Unrepresentable return values
 - Result doesn't fit into the declared return value
- ☐ Null Object References
 - Calling methods on a null object

<#>

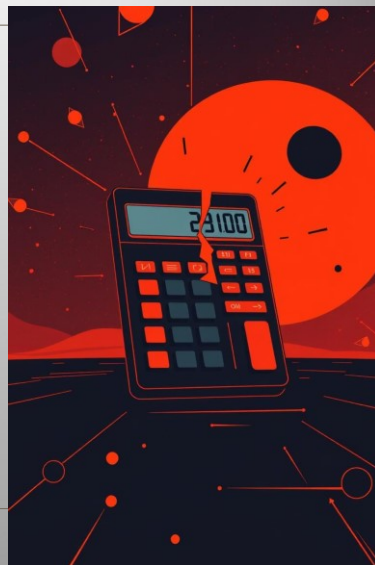


11

Primitive Type Traps

- ☐ Java's primitive types have corner cases that don't behave like true numbers leading to unchecked errors.
- ☐ Integer Division
 - Truncate functions ($5/2 = 2$), part after the decimal is truncated often producing wrong answers
- ☐ Integer Overflow
 - Computations exceed finite ranges quietly wrapping around to incorrect values
- ☐ Special Float Values
 - Operations produces, NAN, NEGATIVE_INFINITY, POSITIVE_INFINITY instead of errors.

<#>



12

Overflow / Wrapping Error

```
public class IntegerOverflowExample {
    public static void main(String[] args) {
        // Maximum value an int can hold
        int maxInt = Integer.MAX_VALUE;
        System.out.println("Maximum value of int: " + maxInt); // Output: 2147483647
        // Adding 1 to the maximum int value causes an overflow
        int overflowedInt = maxInt + 1;
        System.out.println("maxInt + 1: " + overflowedInt); // Output: -2147483648

        // Another example with large numbers
        int a = 2000000000; // 2 billion
        int b = 2000000000; // 2 billion
        int sum = a + b; // Expected: 4 billion, but overflows
        System.out.println("2000000000 + 2000000000: " + sum); // Output: -294967296
    }
}
```

<#>

13

Arrays Vs Lists: Hailstone Example



Arrays

Fixed-length sequences.

Declared: `int[] a = new int[100];`

Danger: Fixed size can lead to overflow bugs (e.g., `a[i] = n;` if `i` exceeds 99).

Security risk: Overflow bugs cause network breaches.



Lists

Variable-length sequences.

Declared: `List list = new ArrayList();`

Safer: Automatically enlarges to fit data.

Requires object types (e.g., `Integer` not `int`).

<#>

14

Iterating with **for** Loops

- ❑ Simplifying sequence processing.

```
// find the maximum point of a hailstone sequence stored in list
int max = 0;
for (int x : list)
{
    max = Math.max(x, max);
}
```

- ❑ This syntax works for both arrays and lists, streamlining code for finding values like the maximum in a sequence.
- ❑ Benefits (guess any two?)

<#>

15

Methods Structure and Specification

- ❑ Organizing code and documenting assumptions.

```
public class Hailstone {    /**      * Compute a hailstone sequence.      *
@param n Starting number for sequence. Assumes n > 0.
@return hailstone sequence starting with n and ending with 1.    */
public static List hailstoneSequence(int n) {
    // ... method body ...
}
}
```

public: Accessible from anywhere.

static: Method belongs to class, not object; called via `ClassName.methodName()`.

Comments: Crucial for specifying method behavior, especially assumptions like $n > 0$.

<#>

16

Mutating Values Vs Reassigning Values

Variable Reassignment

Changing where a variable's arrow points (e.g., `x = 5`; then `x = 10`;

final keyword: Ensures a variable is assigned once, providing static checking for immutable references.

Value Mutation

Changing contents of a mutable value (e.g., adding to a List, modifying an array element).

Immutability: Values cannot change after creation. A key design principle for safer code.

<#>

17

From Hacking to Engineering

Hacking (Bad)

Writing code before testing.

Keeping details in your head.

Assuming no bugs or easy fixes.

Engineering (Good)

Test-first programming.

Documenting assumptions.

Defending against errors (static checking helps!).

Software engineering prioritizes communication with both computers and people for robust, maintainable code.

<#>

18

Primary Goal of this Course

- ☐ To Produce software that is:
 - Safe from bugs
 - ☐ Correctness (Correct behavior right now)
 - ☐ Defensiveness (Correct behavior in future)
 - Easy to understand (Maintainable)
 - Ready for change
 - ☐ Some designs easy to change others require throwing away and rewriting a lot of code.

<#>

19

Starting Java

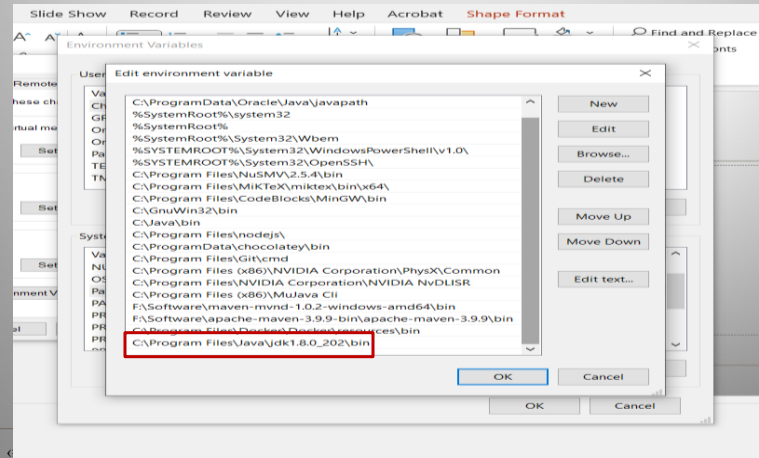
- ☐ Get hold of the latest stable version of Java (>17) and install on your systems.
 - Set the path variable in environment variables to include the path of the bin folder in Java installation directory on your system.
 - E.g., path variable should have an entry like:
 - ☐ C:\Program Files\Java\jdk1.8.0_202\bin
 - Now open a terminal in your OS and test the following commands
 - ☐ javac
 - ☐ java
 - If you see any output then probably you have installed Java on your system properly
 - Otherwise if you see a message like unrecognized command then either a reinstall or setting the path variable properly is required.

<#>

20

Starting Java

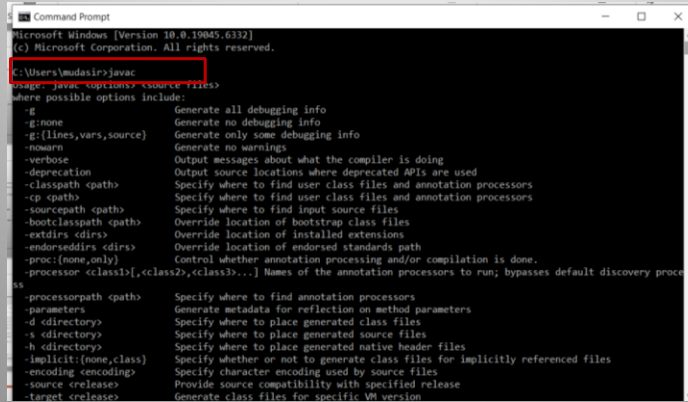
- Setting the path environment variable



21

Starting Java

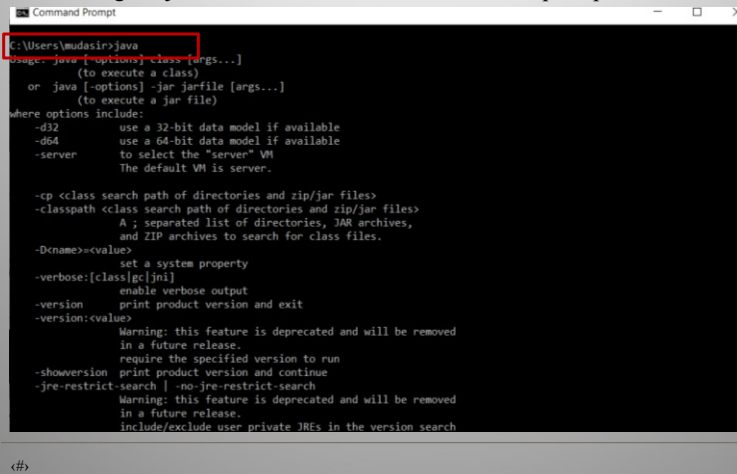
- Running the javac command on terminal / command prompt



22

Starting Java

□ Running the java command on terminal / command prompt



```
Command Prompt
C:\Users\mudasir>java
Usage: java [-options] -class [args...]
           (to execute a class)
or java [-options] -jar jarfile [args...]
           (to execute a jar file)
where options include:
    -d32          use a 32-bit data model if available
    -d64          use a 64-bit data model if available
    -server       to select the "server" VM
                  The default VM is server.

    -cp <class search path of directories and zip/jar files>
    -classpath <class search path of directories and zip/jar files>
                  A ; separated list of directories, JAR archives,
                  and ZIP archives to search for class files.
    -Dname=value  set a system property
    -verbose:[class|gc|jni]
                  enable verbose output
    -version      print product version and exit
    -version:value
                  Warning: this feature is deprecated and will be removed
                  in a future release.
                  require the specified version to run
    -showversion  print product version and continue
    -jre-restrict-search | -no-jre-restrict-search
                  Warning: this feature is deprecated and will be removed
                  in a future release.
                  include/exclude user private JREs in the version search

<#>
```